

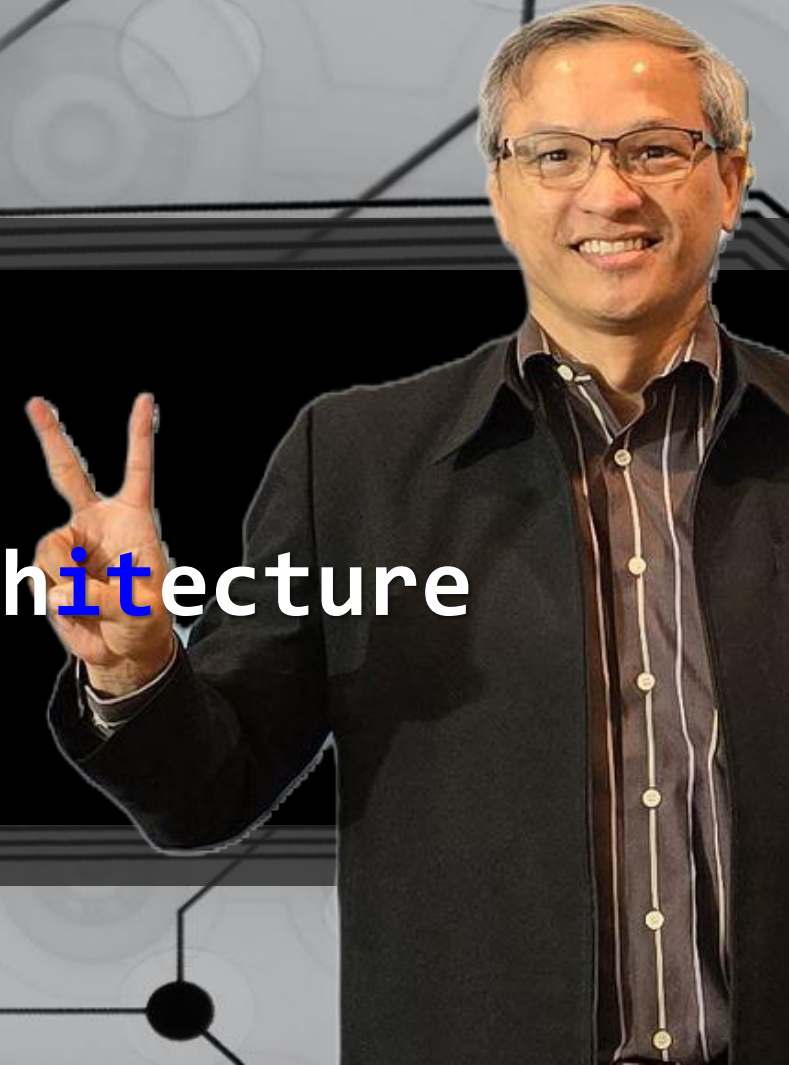


COS2108 – Chapter 1

โครงสร้างและสถาปัตยกรรมคอมพิวเตอร์

Computer Organization and Architecture

ผศ.ชาตรี นิลน้ำเพชร



ข้อมูลทั่วไป – General Information

- รหัสและชื่อวิชา
 - COS2108 (3-0-6)
โครงสร้างและสถาปัตยกรรมคอมพิวเตอร์ Computer Organization and Architecture
- คำอธิบายรายวิชา (Course Description)
 - ศึกษาโครงสร้างและสถาปัตยกรรมของคอมพิวเตอร์ ระบบตัวเลขและการคำนวณ ตรรกะดิจิทัล หน่วยประมวลผลกลาง ชุดคำสั่ง หน่วยความจำ การเชื่อมต่อ และประสิทธิภาพของคอมพิวเตอร์
Study Computer Organization and Architecture, Number Systems and Arithmetic, Digital Logic, Central Processing Unit, Instruction Sets, Memory, Interface, and Computer Performance.
- ชั้นเรียน Google Classroom
 - ช่องทางการเข้าชั้นเรียน, วัน – เวลาเรียน และห้องเรียน, ตำราและเอกสารนำเสนอ, การประเมินผล

<https://qr.page/g/3NGklIFmKM7>



หัวข้อบรรยาย - Topic

- Number Systems
 - Decimal System
 - Binary System
 - Hexadecimal System
 - Conversion
 - Integer
 - Fraction
- Computer Arithmetic
 - Arithmetic and Logic Unit
 - Integer Representation (Fixed Point)
 - Sign Magnitude
 - Twos Complement
 - Range Extension
 - Integer Arithmetic
 - Negation
 - Addition and Subtraction
 - Floating-point Representation
 - IEEE 754 Standard
 - Shift Operations

Decimal System

- Integer (Fixed Point) Example

- $72 = (7 \times 10) + (2 \times 1)$
- $9034 = (9 \times 1000) + (0 \times 100) + (3 \times 10) + (4 \times 1)$

- Base or Radix = 10

- $72 = (7 \times 10^1) + (2 \times 10^0)$
- $9034 = (9 \times 10^3) + (0 \times 10^2) + (3 \times 10^1) + (4 \times 10^0)$

- Fraction Example

- $0.508 = (5 \times 10^{-1}) + (0 \times 10^{-2}) + (8 \times 10^{-3})$

- Integer and Fraction Example

- $9034.508 = (9 \times 10^3) + (0 \times 10^2) + (3 \times 10^1) + (4 \times 10^0) + (5 \times 10^{-1}) + (0 \times 10^{-2}) + (8 \times 10^{-3})$

MSD

LSD

MSD = Most Significant Digit
LSD = Least Significant Digit

Binary System

- Notation

- $0_2 = 0_{10}$
- $1_2 = 1_{10}$

- Base or Radix = 2

- $10_2 = (1 \times 2^1) + (0 \times 2^0) = 2_{10} = 2$
- $1001_2 = (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 9_{10} = 9$

- Fraction Example

- $0.101_2 = (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) = 0.625_{10} = 0.625$

- Integer and Fraction Example

- $1001.101_2 = (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) = 9.625$



MSB = Most Significant Bit
LSB = Least Significant Bit
Bit = Binary Digit

2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
32	16	8	4	2	1	0.5	0.25	0.125	0.0625
	1	0	1	1	0	0	1	1	

Example

$$10110.011_2 = 16 + 4 + 2 + 0.25 + 0.125 = 22.375$$

Hexadecimal System

4 bit = 1 nibble

- Notation

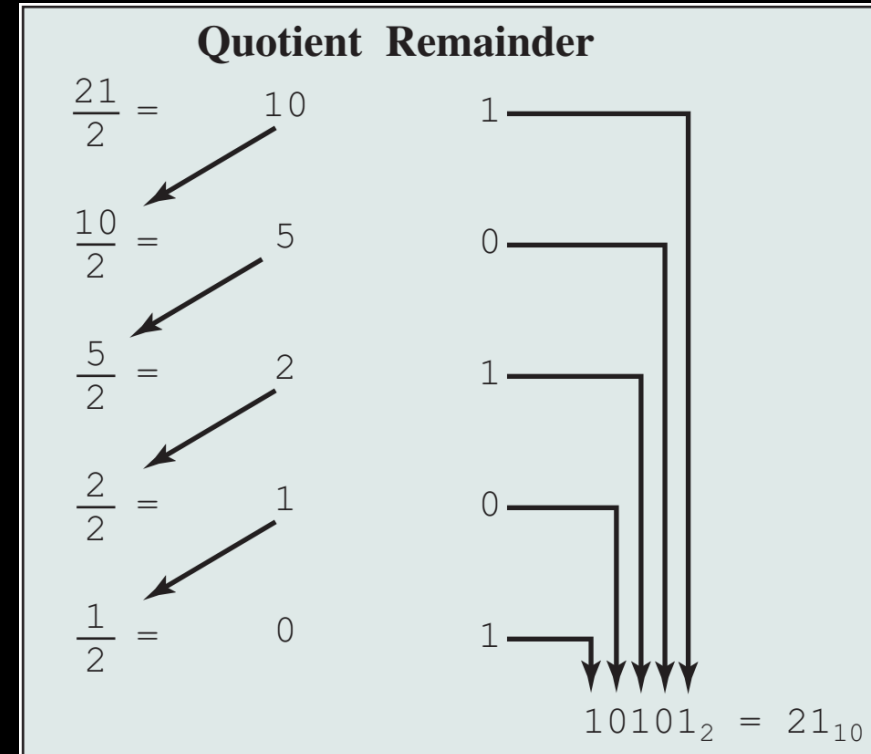
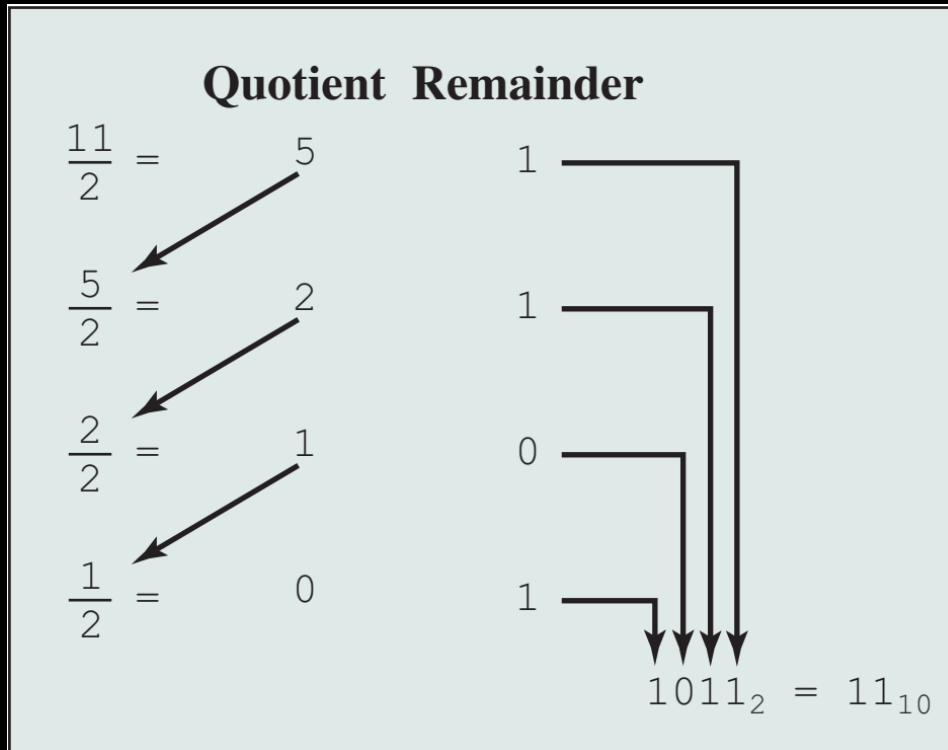
$0000_2 = 0_{16} = 0$	$0100_2 = 4_{16} = 4$	$1000_2 = 8_{16} = 8$	$1100_2 = C_{16} = 12$
$0001_2 = 1_{16} = 1$	$0101_2 = 5_{16} = 5$	$1001_2 = 9_{16} = 9$	$1101_2 = D_{16} = 13$
$0010_2 = 2_{16} = 2$	$0110_2 = 6_{16} = 6$	$1010_2 = A_{16} = 10$	$1110_2 = E_{16} = 14$
$0011_2 = 3_{16} = 3$	$0111_2 = 7_{16} = 7$	$1011_2 = B_{16} = 11$	$1111_2 = F_{16} = 15$

- Integer and Fraction Example

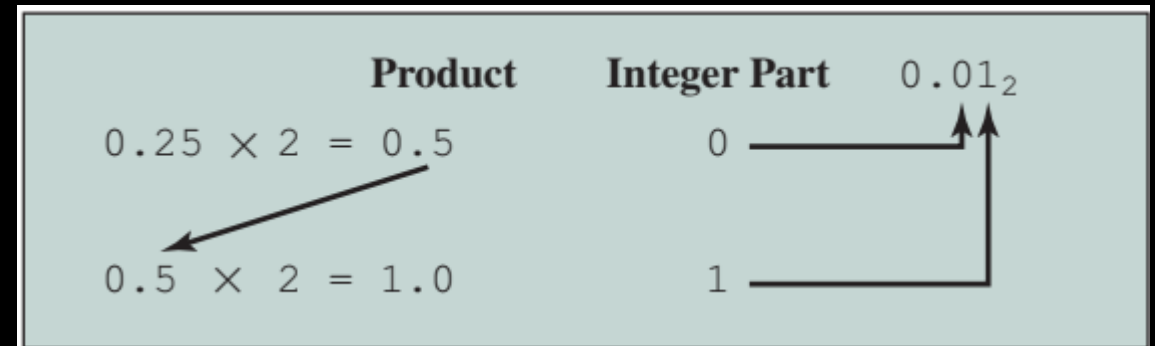
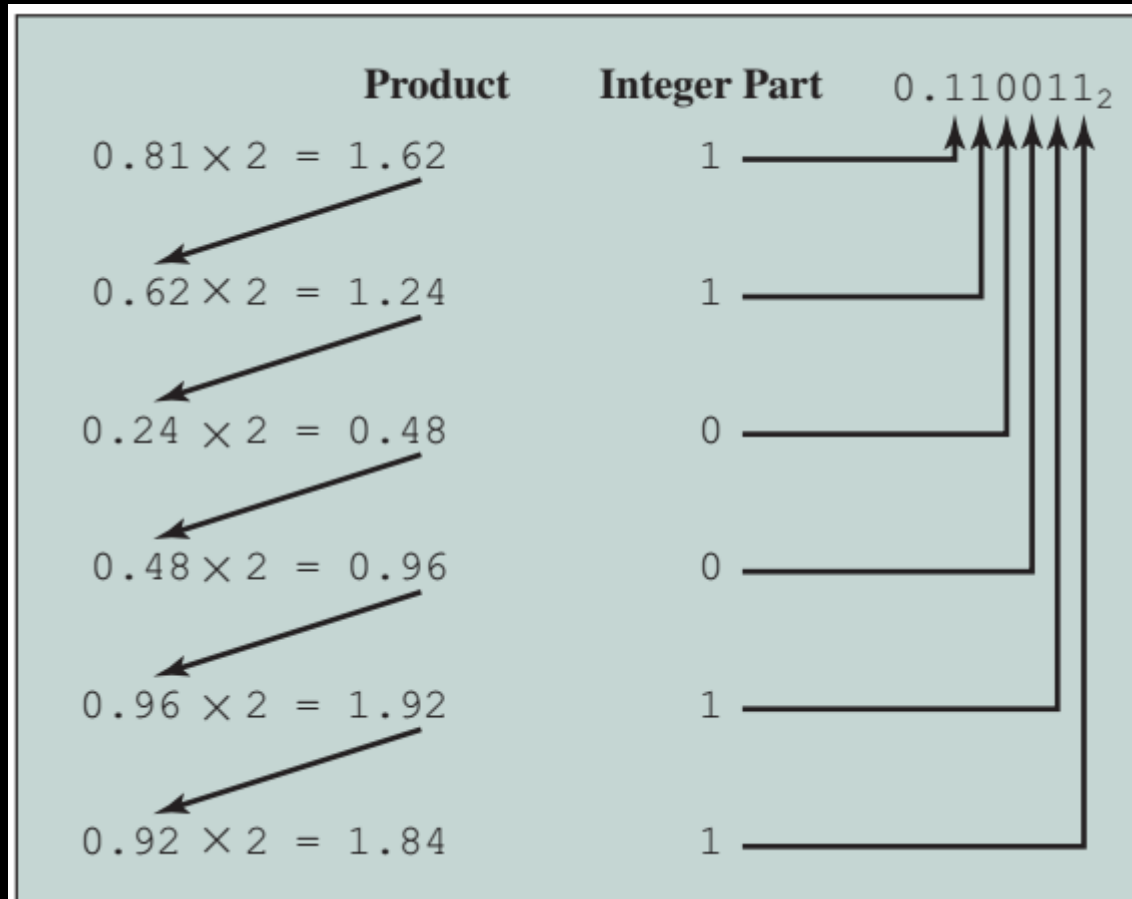
- $3FoA.B0D_{16} = 0011\ 1111\ 0000\ 1010 . 1011\ 0000\ 1101_2$

$$\begin{aligned} &= (3 \times 16^3) + (F \times 16^2) + (0 \times 16^1) + (A \times 16^0) + (B \times 16^{-1}) + (0 \times 16^{-2}) + (D \times 16^{-3}) \\ &= (3 \times 16^3) + (15 \times 16^2) + (0 \times 16^1) + (10 \times 16^0) + (11 \times 16^{-1}) + (0 \times 16^{-2}) + (13 \times 16^{-3}) \\ &= 16138.69067 \end{aligned}$$

Conversion - Integer



Conversion - Fraction



Problems

- แปลงเลขต่อไปนี้ให้อยู่ในรูปแบบเลขฐาน 10

- 101101.001_2
- $BF.93_{16}$

- แปลงเลขต่อไปนี้ให้อยู่ในรูปแบบเลขฐาน 2

- 39.46
- 128.0625

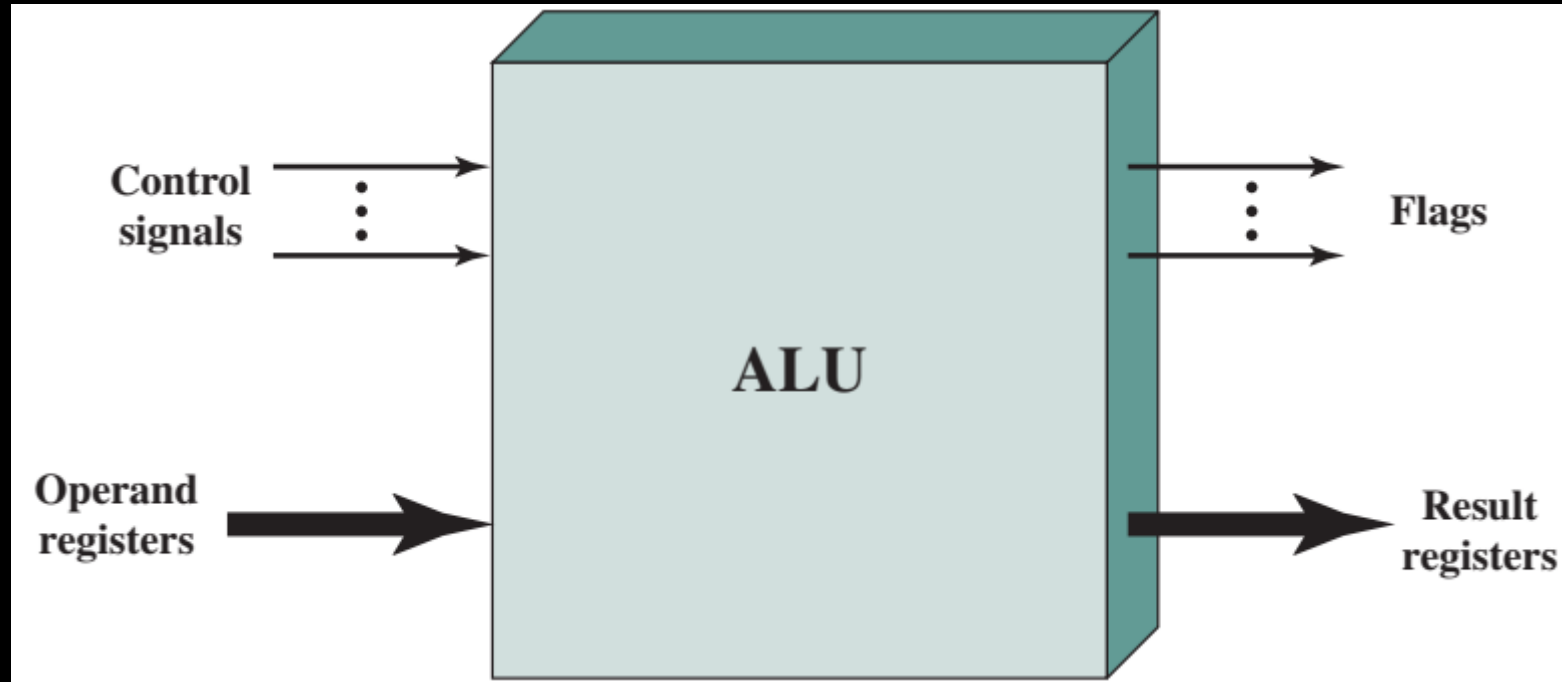
- แปลงเลขฐาน 16 ต่อไปนี้ให้เป็นเลขฐาน 2

- $E5.0A_{16}$
- $18F.BD_{16}$
- $4C3.9_{16}$

- แปลงเลขฐาน 2 ต่อไปนี้ให้อยู่ในรูปแบบเลขฐาน 16

- 10 0111.110₂
- 1100 1100 1110.0011₂
- 1 0101 0101.0101 0101₂

Arithmetic and Logic Unit



Sign Magnitude

- วิธีการจัดเก็บจำนวนเต็มมีเครื่องหมายจะต้องใช้บิตที่มีนัยสำคัญสูงสุด (Most significant bit, MSB) สำหรับจัดเก็บเครื่องหมาย โดยเลข 0_2 แทนเครื่องหมายบวก และเลข 1_2 แทนเครื่องหมายลบ
- กรณีเก็บข้อมูลลงใน 4 Bit memory จะเหลือพื้นที่เก็บจำนวนสัมบูรณ์ (Absolute value) เพียง 3 Bit เท่านั้น



0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111	Memory
0	1	2	3	4	5	6	7	-0	-1	-2	-3	-4	-5	-6	-7	Value

ช่วงของตัวเลขจะมีค่าตั้งแต่ $-(2^{n-1}-1)$ ถึง $+(2^{n-1}-1)$

หมายเหตุ มีเลขศูนย์สองค่าคือ “0” และ “-0”

+22	0001 0110 ₂	+0	0000 0000 ₂
-22	1001 0110 ₂	-0	1000 0000 ₂

Twos Complement

- มีการใช้เลข 0 ตัวเดียว นิยมใช้ในการคำนวณของระบบเลขแบบมีเครื่องหมายในระบบคอมพิวเตอร์อย่างแพร่หลาย เนื่องจากสะดวกต่อการนำมาใช้บวกหรือลบ
- วิธีการจัดเก็บจำนวนเต็มมีเครื่องหมายจะต้องใช้บิตที่มีนัยสำคัญสูงสุด (Most significant bit, MSB) สำหรับจัดเก็บเครื่องหมาย โดยเลข 0₂ แทนเครื่องหมายบวก และเลข 1₂ แทนเครื่องหมายลบ

Sign (-)  Two's Complement Value (-5)

0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111	Memory
0	1	2	3	4	5	6	7	-8	-7	-6	-5	-4	-3	-2	-1	Value

ช่วงของตัวเลขจะมีค่าตั้งแต่ $-(2^{n-1})$ ถึง $+(2^{n-1}-1)$

หมายเหตุ มีเลขศูนย์ค่าเดียวคือ “0”

+22	0001 0110 ₂	0	0000 0000 ₂
-22	1110 1010 ₂	-128	1000 0000 ₂

Binary Adding

- การบวกเลขฐาน 2 ขนาด 1 bit
แบบไม่มีตัวทด

	0
+	0
	<u>0</u>
	0

	0
+	1
	<u>1</u>
	1

	1
+	0
	<u>1</u>
	1

	1
+	1
	<u>1</u>
	10

แบบมีตัวทด

1	
+	0
	<u>0</u>
	1

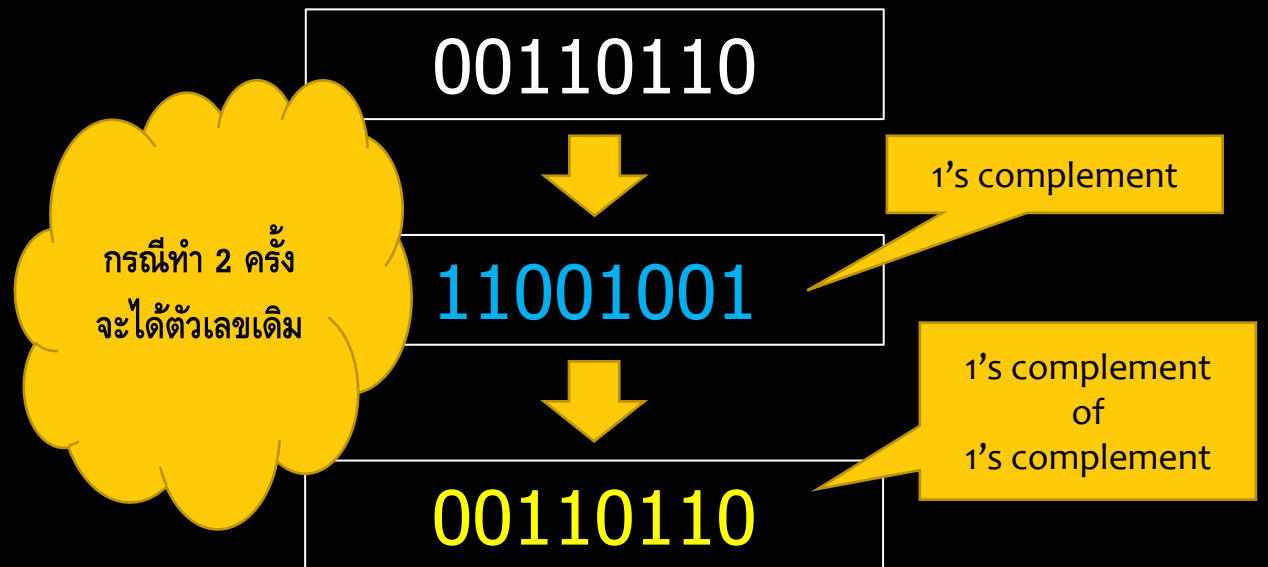
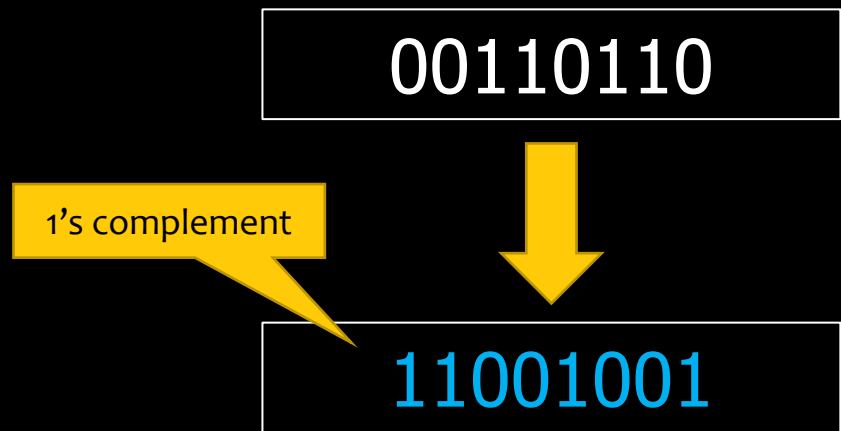
1	
+	0
	<u>1</u>
	10

1	
+	1
	<u>0</u>
	10

1	
+	1
	<u>1</u>
	11

One's complement

- เปลี่ยนเลขปกติให้เป็น 1's complement ได้โดยการสลับ (flip) ตัวเลข โดย Bit ที่มีค่าเป็น "0" ให้สลับเป็น "1" และ Bit ที่มีค่าเป็น "1" ให้สลับเป็น "0"



Two's complement

- เปลี่ยนเลขปกติให้เป็น 2's complement ได้โดยทำให้เป็นเลข 1's complement ก่อน แล้วนำผลลัพธ์ที่ได้ไปบวก "1"



Two's complement

- วิธีทำได้โดย
 - สำเนาทุก bit จากขวาไปซ้ายจนกระทั่งพบ bit ที่เป็น “1” แล้วให้ทำการสลับ (flip) bit ที่เหลือทุก bit จนหมด

00110110



11001010

2's complement

11011000



00101000

2's complement

10011011



01100101

2's complement



4-Bit Representations

Decimal Representation	Sign-Magnitude Representation	Twos Complement Representation	Biased Representation
+8	—	—	1111
+7	0111	0111	1110
+6	0110	0110	1101
+5	0101	0101	1100
+4	0100	0100	1011
+3	0011	0011	1010
+2	0010	0010	1001
+1	0001	0001	1000
+0	0000	0000	0111
−0	1000	—	—
−1	1001	1111	0110
−2	1010	1110	0101
−3	1011	1101	0100
−4	1100	1100	0011
−5	1101	1011	0010
−6	1110	1010	0001
−7	1111	1001	0000
−8	—	1000	—

Range Extension

+18 = 0001 0010	Sign Magnitude, 8 bits
+18 = 0000 0000 0001 0010	Sign Magnitude, 16 bits
-18 = 1001 0010	Sign Magnitude, 8 bits
-18 = 1000 0000 0001 0010	Sign Magnitude, 16 bits
+18 = 0001 0010	Twos Complement, 8 bits
+18 = 0000 0000 0001 0010	Twos Complement, 16 bits
-18 = 1110 1110	Twos Complement, 8 bits
-18 = 1111 1111 1110 1110	Twos Complement, 16 bits

Negation

Sign Magnitude	Original (8 bits)	Negation (8 bits)
+18	0001 0010	1001 0010
-18	1001 0010	0001 0010

Twos Complement	Original (8 bits)	Negation (8 bits)
+18	0001 0010	1110 1110
-18	1110 1110	0001 0010
0	0000 0000	0000 0000
-128	1000 0000	1000 0000

Addition

$\begin{array}{r} 1001 = -7 \\ + \underline{0101} = 5 \\ 1110 = -2 \end{array}$ (a) $(-7) + (+5)$	$\begin{array}{r} 1100 = -4 \\ + \underline{0100} = 4 \\ \textcolor{teal}{1}0000 = 0 \end{array}$ (b) $(-4) + (+4)$
$\begin{array}{r} 0011 = 3 \\ + \underline{0100} = 4 \\ 0111 = 7 \end{array}$ (c) $(+3) + (+4)$	$\begin{array}{r} 1100 = -4 \\ + \underline{1111} = -1 \\ \textcolor{teal}{1}1011 = -5 \end{array}$ (d) $(-4) + (-1)$
$\begin{array}{r} 0101 = 5 \\ + \underline{0100} = 4 \\ 1001 = \text{Overflow} \end{array}$ (e) $(+5) + (+4)$	$\begin{array}{r} 1001 = -7 \\ + \underline{1010} = -6 \\ \textcolor{teal}{1}0011 = \text{Overflow} \end{array}$ (f) $(-7) + (-6)$

Subtraction

$$A - B = A + (-B) = A + (\bar{B} + 1)$$

$\begin{array}{r} 0010 = 2 \\ +1001 = -7 \\ \hline 1011 = -5 \end{array}$ (a) M = 2 = 0010 S = 7 = 0111 -S = 1001	$\begin{array}{r} 0101 = 5 \\ +1110 = -2 \\ \hline 10011 = 3 \end{array}$ (b) M = 5 = 0101 S = 2 = 0010 -S = 1110
$\begin{array}{r} 1011 = -5 \\ +1110 = -2 \\ \hline 11001 = -7 \end{array}$ (c) M = -5 = 1011 S = 2 = 0010 -S = 1110	$\begin{array}{r} 0101 = 5 \\ +0010 = 2 \\ \hline 0111 = 7 \end{array}$ (d) M = 5 = 0101 S = -2 = 1110 -S = 0010
$\begin{array}{r} 0111 = 7 \\ +0111 = 7 \\ \hline 1110 = \text{Overflow} \end{array}$ (e) M = 7 = 0111 S = -7 = 1001 -S = 0111	$\begin{array}{r} 1010 = -6 \\ +1100 = -4 \\ \hline 10110 = \text{Overflow} \end{array}$ (f) M = -6 = 1010 S = 4 = 0100 -S = 1100

Arithmetic operations example

- หาผลบวกของเลขสองจำนวน

$A = 00010001_2$ และ $B = 00010110_2$

				1					Carry
	0	0	0	1	0	0	0	1	A
+	0	0	0	1	0	1	1	0	B
	0	0	1	0	0	1	1	1	R

$$(+17) + (+22) = (+39)$$

Arithmetic operations example

- หาผลบวกของเลขสองจำนวน

$A = 00011000_2$ และ $B = 11101111_2$

	1	1	1	1	1				Carry
	0	0	0	1	1	0	0	0	A
+	1	1	1	0	1	1	1	1	B
	0	0	0	0	0	1	1	1	R

$$(+24) + (-17) = (+7)$$

Arithmetic operations example

- หาผลลบของเลขสองจำนวน โดยลบ B ออกจาก A

$A = 00011000_2$ และ $B = 11101111_2$

				1					Carry
	0	0	0	1	1	0	0	0	A
+	0	0	0	1	0	0	0	1	$\overline{B} + 1$
	0	0	1	0	1	0	0	1	R

$$(+24) - (-17) = (+41)$$

Arithmetic operations example

- หาผลลบของเลขสองจำนวน โดยลบ B ออกจาก A

$$A = 11011101_2 \text{ และ } B = 00010100_2$$

1	1	1	1	1	1				Carry
	1	1	0	1	1	1	0	1	A
+	1	1	1	0	1	1	0	0	$\overline{B} + 1$
	1	1	0	0	1	0	0	1	R

$$(-35) - (+20) = (-55)$$

Arithmetic operations example

- หาผลบวกของเลขสองจำนวน

$A = 0111111_2$ และ $B = 0000011_2$

	1	1	1	1	1	1	1	Carry
	0	1	1	1	1	1	1	A
+	0	0	0	0	0	0	1	B
	1	0	0	0	0	0	1	R

-126

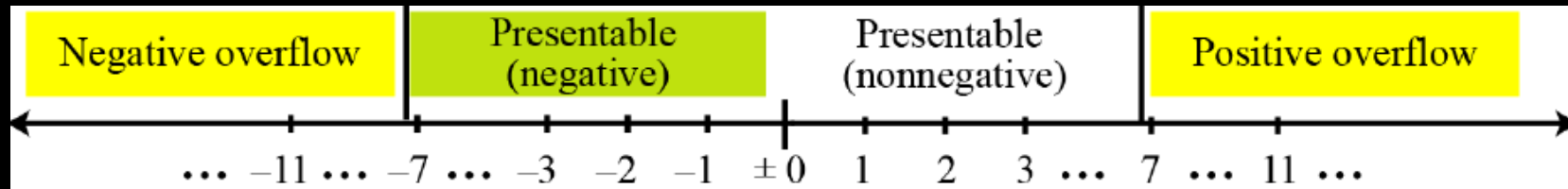
$$(+127) + (+3) = (+130)$$

Error

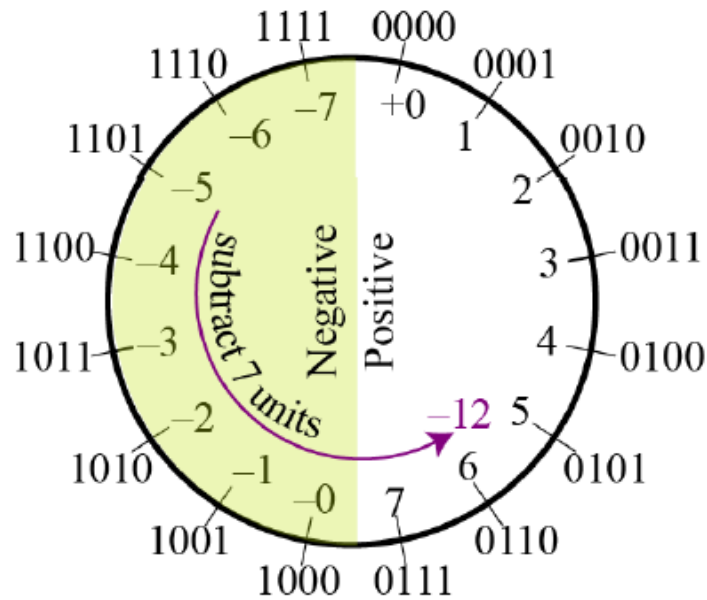
Overflow: Result not in the range -128 to 127

Warning: Each number and result should be in the range defined by the bit allocation.

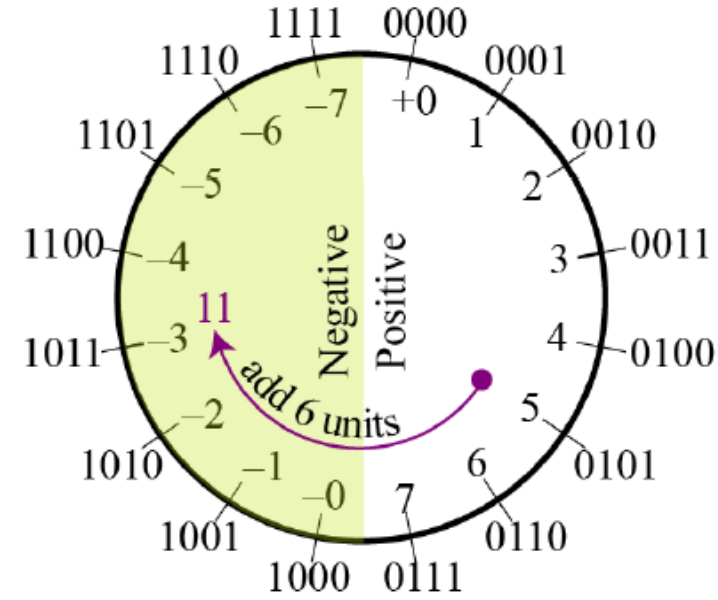
Sign Magnitude - Overflow



a. Linear characteristic of an integer in sign-and-magnitude format

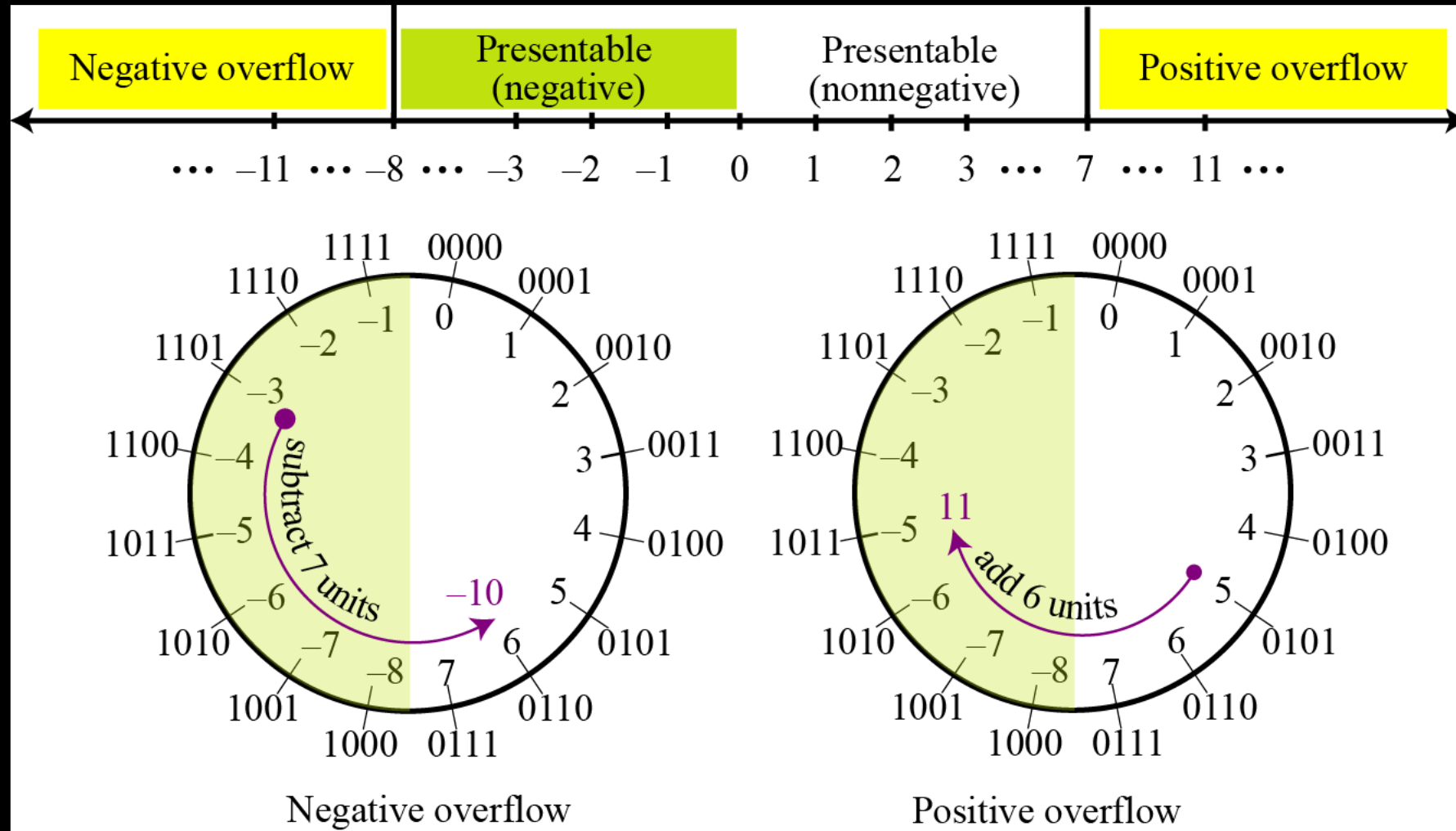


b. Negative overflow



c. Positive overflow

Twos Complement - Overflow



Flowchart and Block Diagram of H/W for Addition and Subtraction

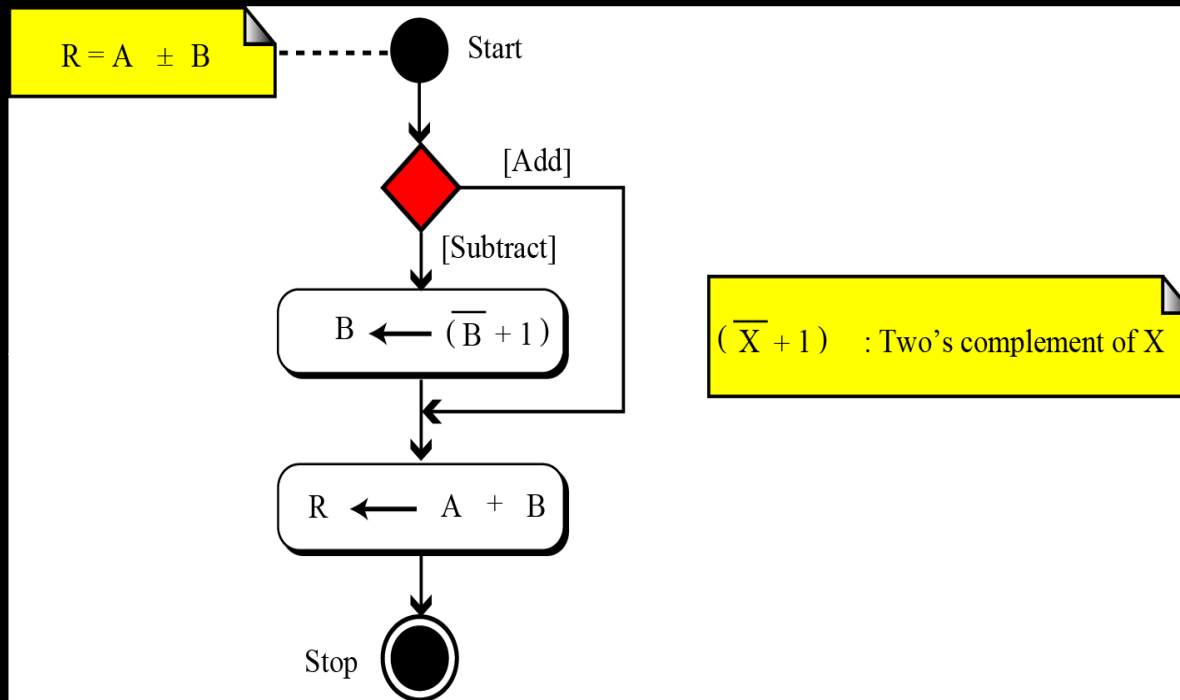
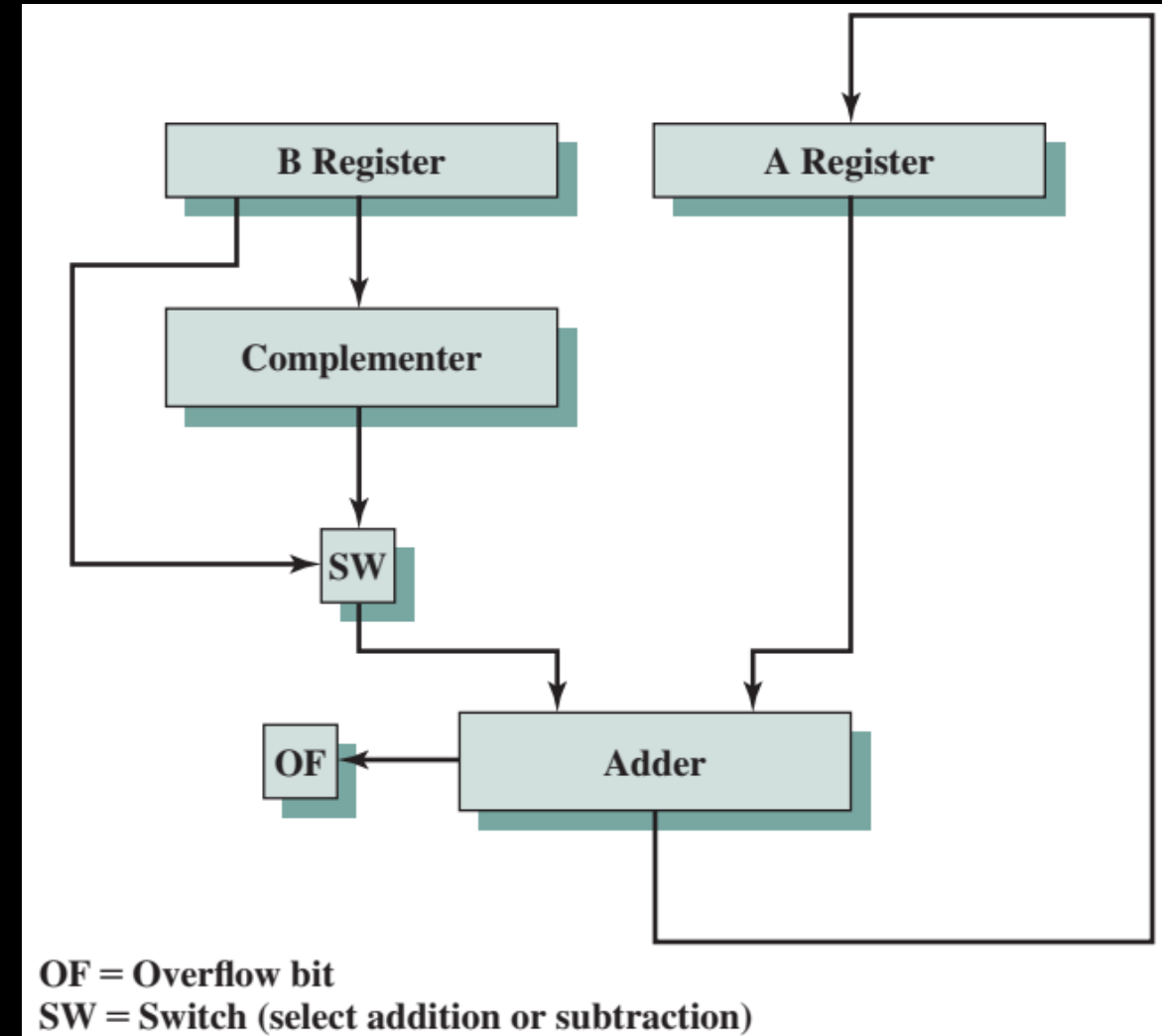


Image Reference:

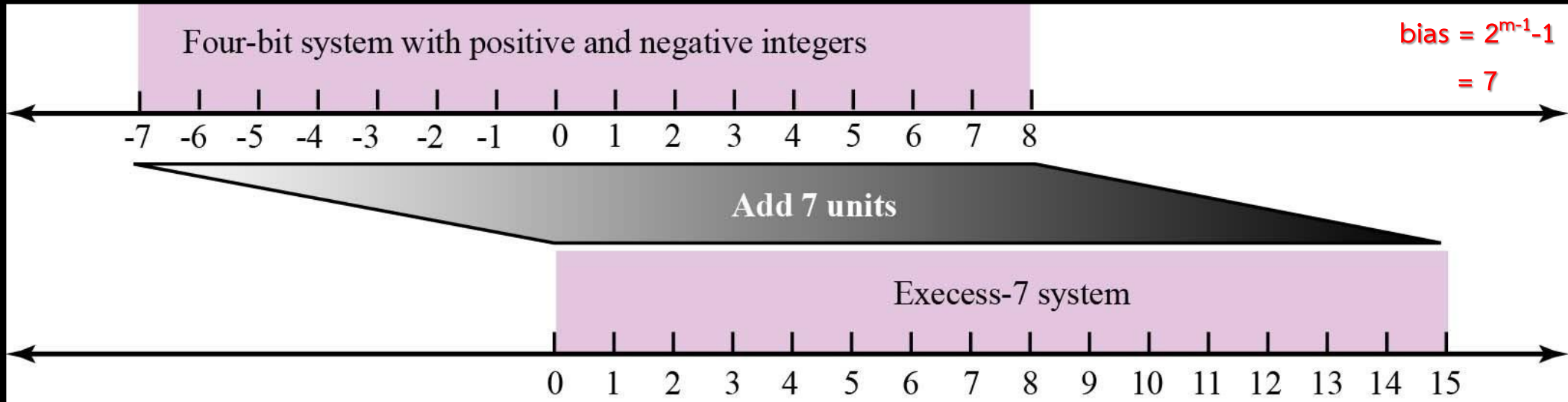


Problems

- จงหาเลข **twos complement** ขนาด **8 bit** ของเลขต่อไปนี้
 - 115
 - -34
- จงทำการคำนวณเลข **twos complement** ขนาด **8 bit** ต่อไปนี้ และดูว่าเกิด **Overflow** หรือ **Zero Flag** หรือไม่ เพราะอะไร
 - $45 + 70$
 - $37 - 50$
 - $-20 + 20$
 - $-46 - 60$
 - $94 + 35$
 - $-86 - 60$

Excess system

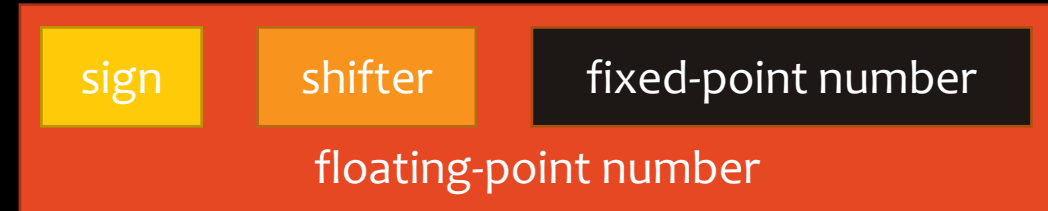
- Excess system จะจัดเก็บตัวเลขจำนวนเต็มแบบมีเครื่องหมายให้อยู่ในรูปเลขจำนวนเต็มแบบไม่มีเครื่องหมาย โดยการนำตัวเลขที่มีค่าบวกจำนวนหนึ่งบวกเพิ่มเข้าไปกับตัวเลขที่ต้องการจัดเก็บ เพื่อให้เลขที่จะถูกนำไปจัดเก็บกลายเป็นค่าที่ไม่ติดลบ เรียกเลขจำนวนที่นำไปบวกเพิ่มนี้ว่า bias



Binary	Unsigned	Sign-and-magnitude	One's complement	Two's complement	Excess-7
0000	0	0	0	0	-7
0001	1	1	1	1	-6
0010	2	2	2	2	-5
0011	3	3	3	3	-4
0100	4	4	4	4	-3
0101	5	5	5	5	-2
0110	6	6	6	6	-1
0111	7	7	7	7	0
1000	8	-0	-7	-8	1
1001	9	-1	-6	-7	2
1010	10	-2	-5	-6	3
1011	11	-3	-4	-5	4
1100	12	-4	-3	-4	5
1101	13	-5	-2	-3	6
1110	14	-6	-1	-2	7
1111	15	-7	-0	-1	8

Floating-Point

- แนวทางที่จะช่วยแก้ปัญหาให้เกิดทั้งความแม่นยำและความเที่ยงตรงคือการจัดเก็บจำนวนจริงโดยใช้ ตัวเลขแบบจุดทศนิยมลอยตัว (floating-point number) ซึ่งจะแบ่งส่วนที่จะจัดเก็บออกเป็น 3 ส่วนคือ
 - เครื่องหมาย (sign)
 - ตัวชี้กำลัง (shifter)
 - ตัวเลขแบบจุดทศนิยมตายตัว (fixed-point number)



Floating-Point Example

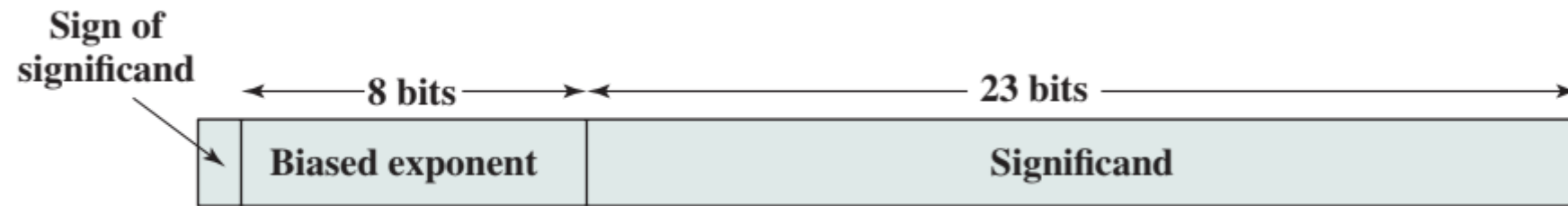
- ต้องการนำเสนอตัวเลขทศนิยมฐาน 10 ด้วยวิธี floating-point number

Actual number	Scientific notation	Sign	Shifter	Fixed-point number
4,321,000,000,000,000,000.00	4.321×10^{21}	+	21	4.321
-0.000000000000001234	-1.234×10^{-14}	-	-14	1.234

- ต้องการนำเสนอตัวเลขทศนิยมฐาน 2 ด้วยวิธี floating-point number

Actual number	Scientific notation	Sign	Shifter	Fixed-point number
-10101110000000000000.00	-1.010111×2^{19}	-	19	1.010111
0.000000000000001010111	1.010111×2^{-14}	+	-14	1.010111

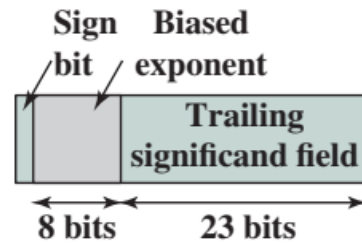
Floating-Point 32 bits



(a) Format

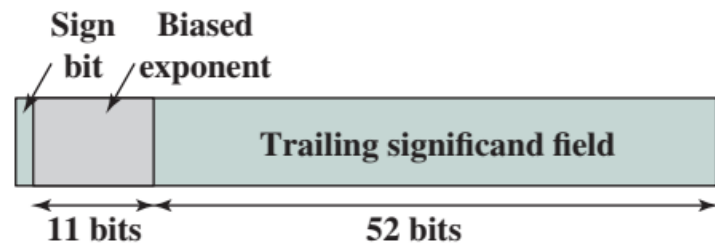
$$\begin{array}{lcl}
 1.1010001 \times 2^{10100} & = & 0 \ 10010011 \ 101000100000000000000000 = 1.6328125 \times 2^{20} \\
 -1.1010001 \times 2^{10100} & = & 1 \ 10010011 \ 101000100000000000000000 = -1.6328125 \times 2^{20} \\
 1.1010001 \times 2^{-10100} & = & 0 \ 01101011 \ 101000100000000000000000 = 1.6328125 \times 2^{-20} \\
 -1.1010001 \times 2^{-10100} & = & 1 \ 01101011 \ 101000100000000000000000 = -1.6328125 \times 2^{-20}
 \end{array}$$

IEEE 754 Format



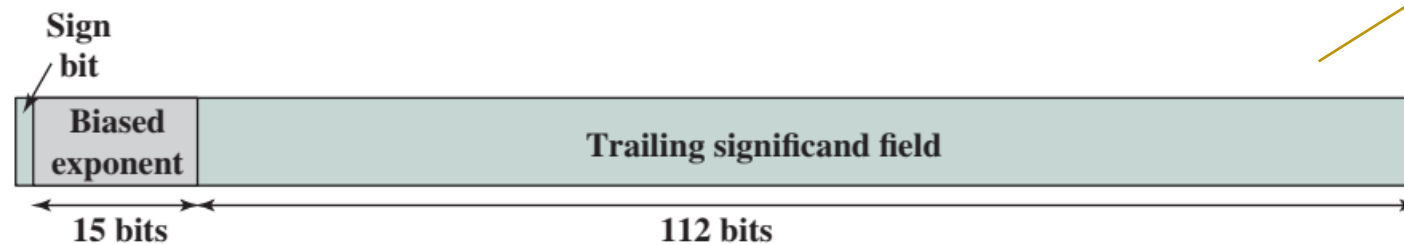
(a) Binary32 format

Single Precision



(b) Binary64 format

Double Precision



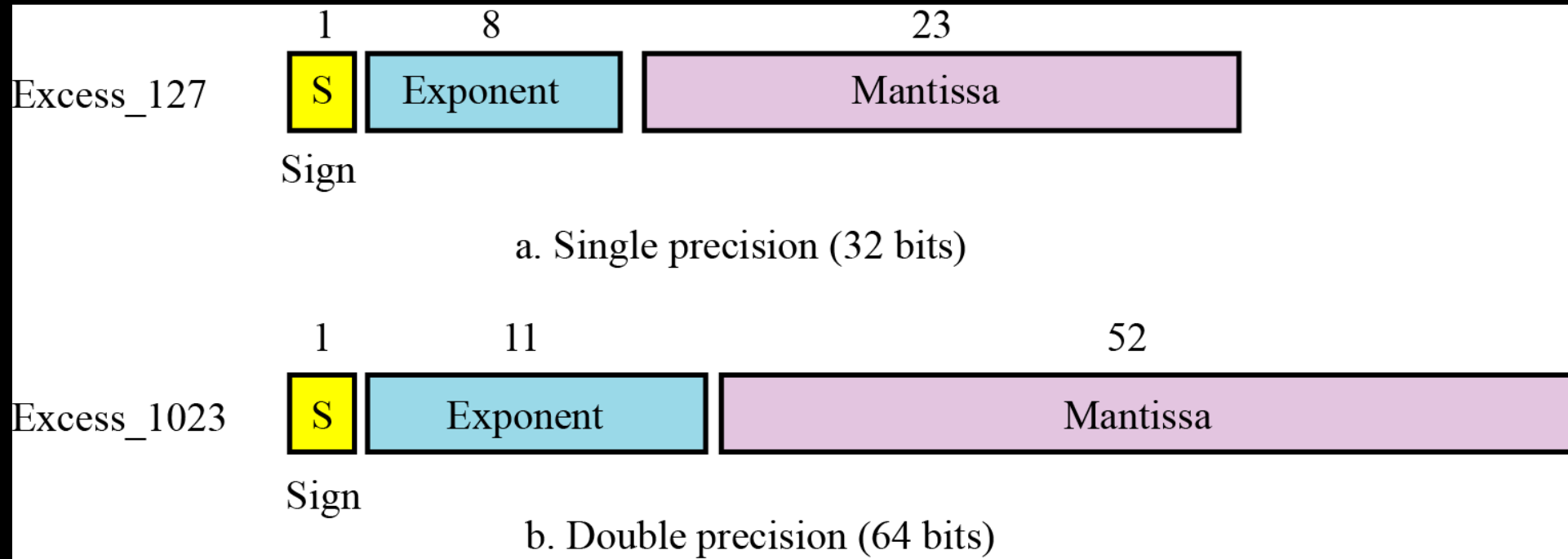
(c) Binary128 format

Quadruple Precision

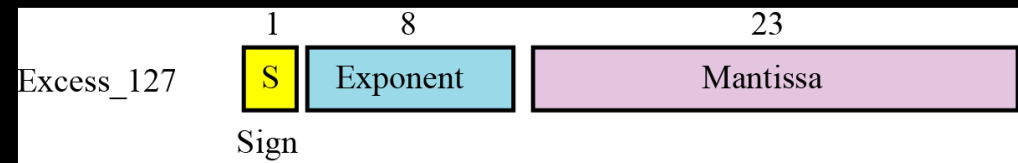
IEEE 754 Parameters

Parameter	Format		
	Binary32	Binary64	Binary128
Storage width (bits)	32	64	128
Exponent width (bits)	8	11	15
Exponent bias	127	1023	16383
Maximum exponent	127	1023	16383
Minimum exponent	-126	-1022	-16382
Approx normal number range (base 10)	$10^{-38}, 10^{+38}$	$10^{-308}, 10^{+308}$	$10^{-4932}, 10^{+4932}$
Trailing significand width (bits)*	23	52	112
Number of exponents	254	2046	32766
Number of fractions	2^{23}	2^{52}	2^{112}
Number of values	1.98×2^{31}	1.99×2^{63}	1.99×2^{128}
Smallest positive normal number	2^{-126}	2^{-1022}	2^{-16362}
Largest positive normal number	$2^{128} - 2^{104}$	$2^{1024} - 2^{971}$	$2^{16384} - 2^{16271}$
Smallest subnormal magnitude	2^{-149}	2^{-1074}	2^{-16494}

IEEE 754 - Floating-Point Standard



ตัวอย่าง การจัดเก็บ



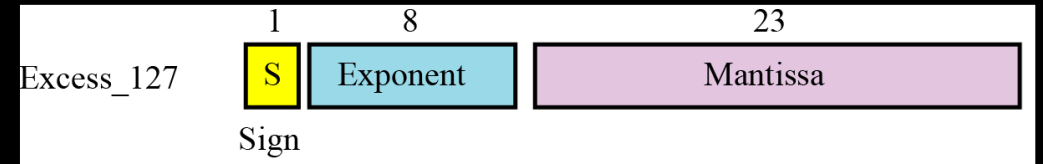
- ต้องการเก็บเลข 5.75 ในรูปแบบ IEEE 754 Single Precision (Excess_127)
 - แปลง $5.75 \rightarrow (101.11)_2$
 - Normalization $(101.11)_2 = (1.0111)_2 \times 2^2$
 - เลขเป็นจำนวนบวก ดังนั้น $S = (0)_2$
 - Exponent $= 2 + 127 = 129 = (10000001)_2$
 - Mantissa $= (0111)_2$
 - เติม 0 ด้านขวาให้ Mantissa จนเต็ม 23 Bit $(01110000000000000000000)_2$

Memory

01000000101110000000000000000000

หมายเหตุ ไม่มีการบันทึกเลข subscript ลงในหน่วยความจำ

ตัวอย่าง การจัดเก็บ



- ต้องการเก็บเลข -161.875 ในรูปแบบ IEEE 754 Single Precision (Excess_127)

- แปลง 161.875 $\rightarrow (10100001.111)_2$

- Normalization $(10100001.111)_2 = (1.0100001111)_2 \times 2^7$

- เลขเป็นจำนวนลบ ดังนั้น $S = (1)_2$

- Exponent = $7 + 127 = 134 = (10000110)_2$

- Mantissa = $(0100001111)_2$

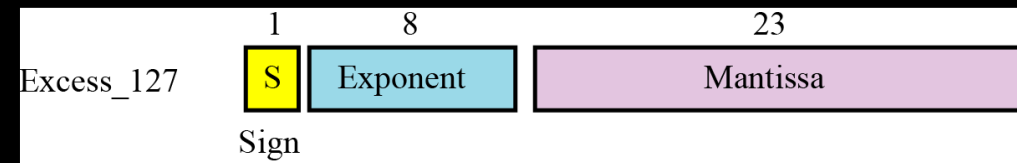
- เติม 0 ด้านขวาให้ Mantissa จนเต็ม 23 Bit $(010000111100000000000000)_2$

Memory

110000110010000111100000000000000

หมายเหตุ ไม่มีการบันทึกเลข subscript ลงในหน่วยความจำ

ตัวอย่าง การจัดเก็บ



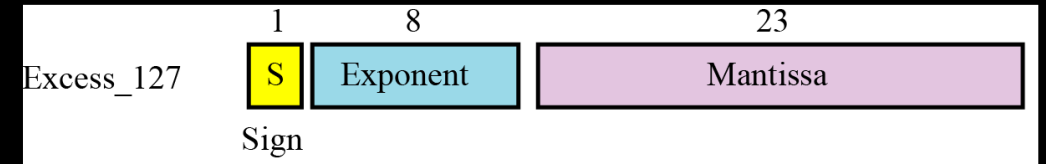
- ต้องการเก็บเลข -0.0234375 ในรูปแบบ IEEE 754 Single Precision (Excess_127)
 - แปลง $0.0234375 \rightarrow (0.0000011)_2$
 - Normalization $(0.0000011)_2 = (1.1)_2 \times 2^{-6}$
 - เลขเป็นจำนวนลบ ดังนั้น $S = (1)_2$
 - Exponent $= -6 + 127 = 121 = (01111001)_2$
 - Mantissa $= (1)_2$
 - เติม 0 ด้านขวาให้ Mantissa จนเต็ม 23 Bit $(10000000000000000000000)_2$

Memory

[illegible]

หมายเหตุ: ไม่มีการบันทึกเลข subscript ลงในหน่วยความจำ

ตัวอย่าง คำนวณ



- คำนวณเลข ($11001010000000000111000100001111$)₂ จาก 32 Bit pattern memory ต้องการทราบว่ามีความเท่าใดในระบบ IEEE 754 Single Precision (Excess_127)
 - $S = (1)_2$ ดังนั้นเลขนี้มีค่าเป็นลบ
 - $E = (10010100)_2 = 148$ ดังนั้น Shifter = $148 - 127 = 21$
 - $M = (00000000111000100001111)_2$
 - หาค่า Normalization ได้เท่ากับ $(1.00000000111000100001111)_2 \times 2^{21}$
 - ซึ่งเท่ากับ $(1000000001110001000011.11)_2 = 2,104,378.75$
 - ดังนั้นผลลัพธ์จะมีความเท่ากับ $-2,104,378.75$

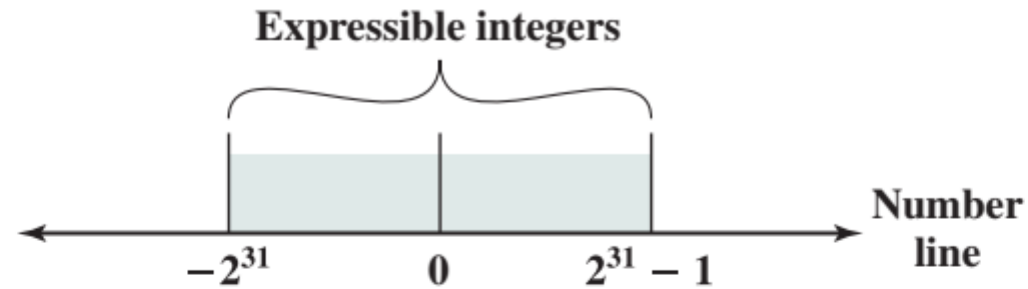
Memory

110010100000000000111000100001111

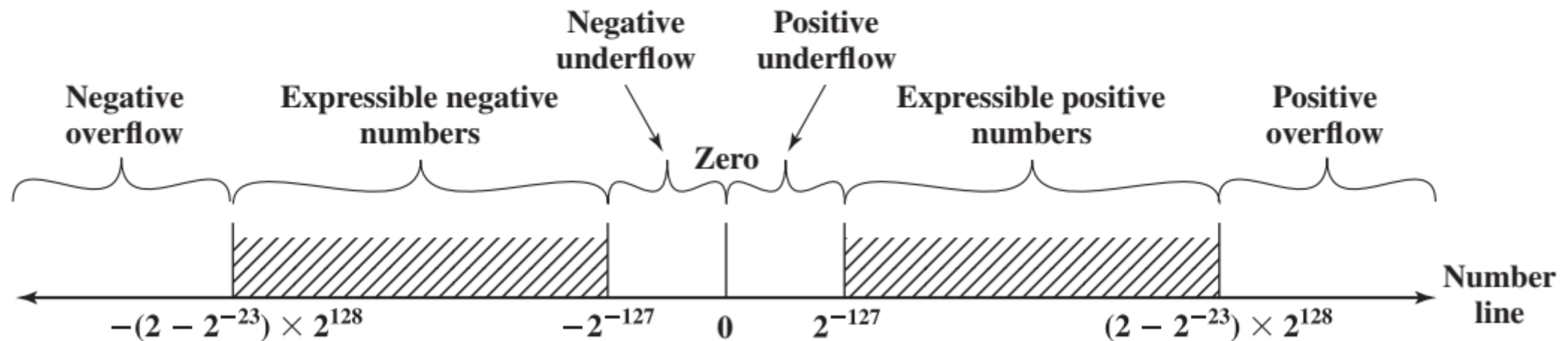
ผลลัพธ์

-2,104,378.75

Expressible Numbers in 32 bits



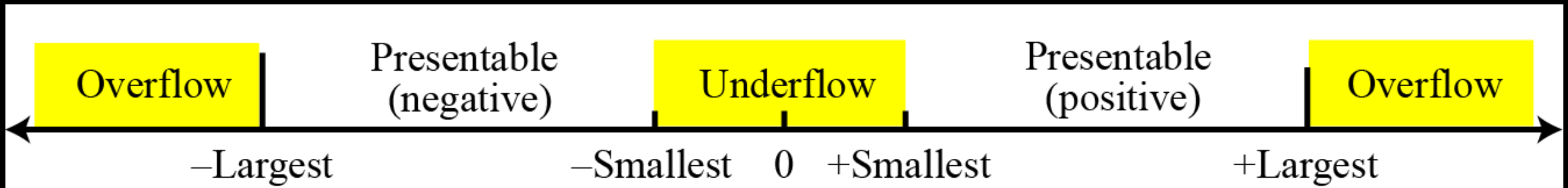
(a) Twos complement integers



(b) Floating-point numbers

Floating-Point - Overflow

- สามารถเกิดได้ทั้ง positive overflow, negative overflow และ underflow



การบันทึกเลข **0.0** ถือเป็นกรณีพิเศษ
โดยให้บันทึกเป็น **0** ทั้งหมดทุก **bit**

Problems

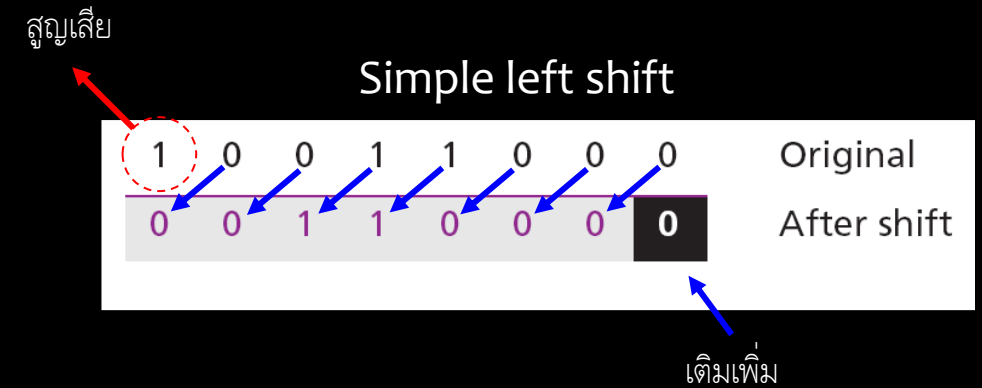
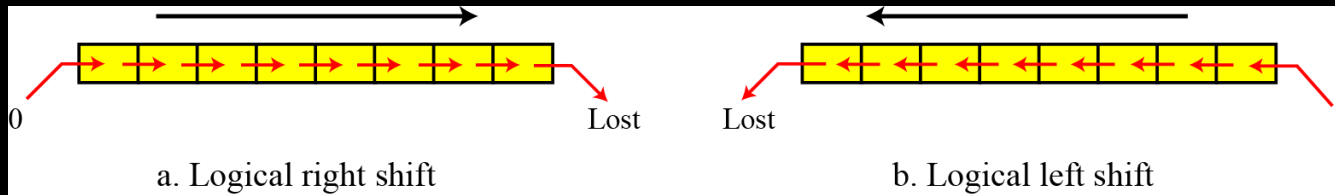
- แปลงเลขต่อไปนี้ให้อยู่ในรูปแบบ IEEE 754 32-bit
 - 32.54
 - 176.89
- แปลงเลขต่อไปนี้ให้อยู่ในรูปแบบ IEEE 754 16-bit
(Half Precision Format -> Sign = 1 bit, Exponent = 5 bit, Fraction = 10 bit)
 - 32.54
 - 176.89
- แปลงเลข IEEE 754 32-bit ต่อไปนี้ให้เป็นเลขฐาน 10
 - 1100 0001 0001 0000 0000 0000 0000 0000
 - 0011 1111 0000 0110 0110 0110 0110 0110

Shift operations

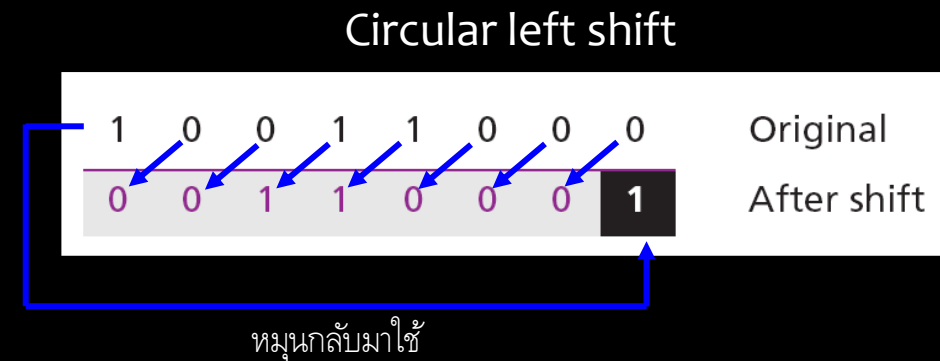
- แบ่งออกเป็น 2 ประเภท
 - Logical shift operations
 - Simple shift
 - Circular shift
 - Arithmetic shift operations

Logical shift operations

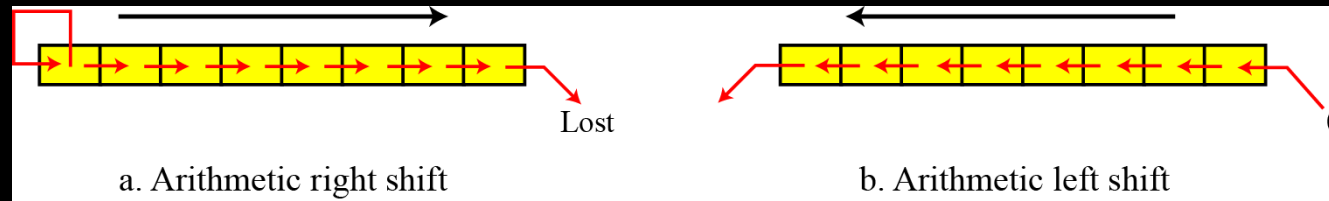
- Simple shift



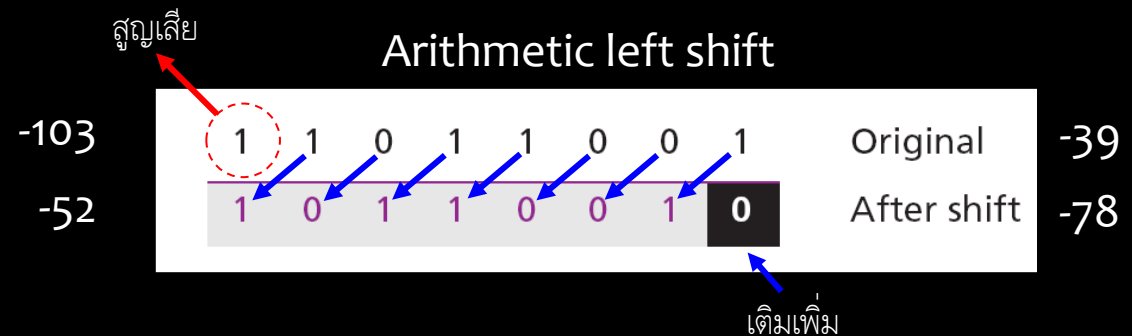
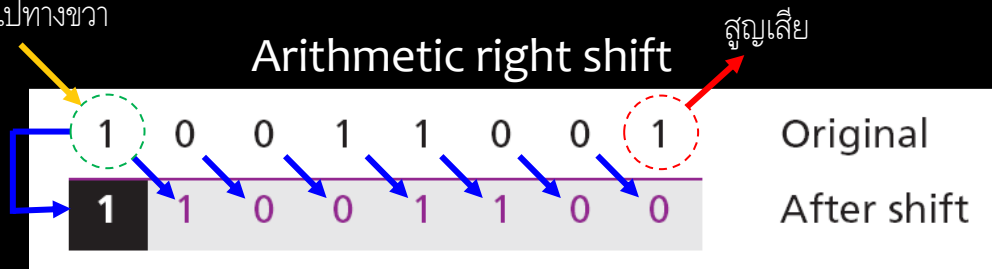
- Circular shift



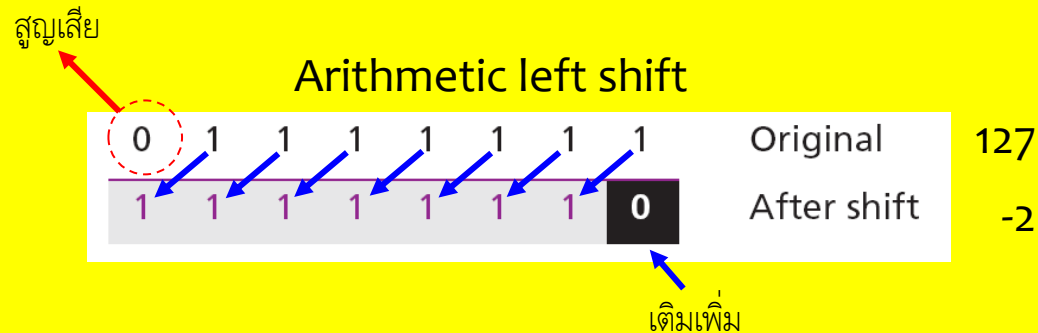
Arithmetic shift operations



อยู่กับที่และ
เลื่อนไปทางขวา



Overflow case



Problems

- จงดำเนินการเลื่อนบิตดังต่อไปนี้
 - Left Shift 3 Bit ของ C3
 - Right Shift 1 Bit ของ 5B
 - Circular Left Shift 4 Bit ของ A6
 - Circular Right Shift 5 Bit ของ 35
 - Arithmetic Left Shift 3 Bit ของ 64
 - Arithmetic Right Shift 1 Bit ของ C4